



MEMBERS

Name	SAP ID
Ayush Negi	590015410
Ayush Kumar	590014539
Aditi Vijay	590018228
Mayank Tanishk	590015863
Mayank Parmar	590017039
Sakshi Singh	590012402
Samiksha Rajan	590016234
Satyam Pandey	590016569



INDEX

Section	Content Description	Page No.
1. . Goal of the Phase	Designing a relational database schema and implementing a JDBC-based DAO layer for the Placement Drive Management System.	3
2. Database Schema Design	Creating tables (Company, Student, Drive, Application, Interview Slot) with proper primary keys and foreign key relationships.	3-5
3. DAO Layer Design	Implementing DAO classes (CompanyDAO, DriveDAO, ApplicationDAO) to handle database operations using JDBC.	5-7
4. CRUD Operations	Performing Create, Read, Update, and Delete operations for core entities like Company, Drive, and Application.	7-10
5. JDBC Implementation	Using PreparedStatement, centralized connection handling, and proper ResultSet mapping while keeping SQL in DAO layer.	11
6. Working Flow of the System	Demonstrating system flow: User → DAO → Database and operational flow from company creation to application storage.	11-12
7. Project Folder Structure	Defining the folder structure including db, dao, model, and main classes along with external libraries.	13-14
8. Design Rules Adherence	Ensuring no SQL in UI, proper layer separation, and maintaining clean architecture.	14-15
9. Common Mistakes Avoided	Avoiding missing keys, improper relationships, scattered SQL, and direct DB access from UI.	15

10. Review Checklist	Verifying schema correctness, DAO functionality, CRUD operations, and complete working flow.	15
-----------------------------	--	-----------

Object Oriented Programming (CSEG1044)

Phase 3: DATABASE AND DAO LAYER

Project Title: Placement Drive Management System.

- **Goal of the Phase:** The primary goal of this phase is to design a relational database schema and implement a JDBC-based DAO (Data Access Object) layer for the Placement Drive Management System.
- This phase ensures that:
- Data is stored in a structured format
- Relationships between entities are properly maintained
- Database operations are handled efficiently using Java

2. Database Schema Design:

- The system consists of the following core tables:
- **Company**

```
CREATE TABLE companies (  
    company_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    location VARCHAR(100)  
);
```

- **Student**

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    email VARCHAR(100)  
);
```

- **Drive**

```
CREATE TABLE drives (  
    drive_id INT PRIMARY KEY AUTO_INCREMENT,  
    company_id INT,  
    date DATE,  
    FOREIGN KEY (company_id) REFERENCES  
    companies(company_id)  
);
```

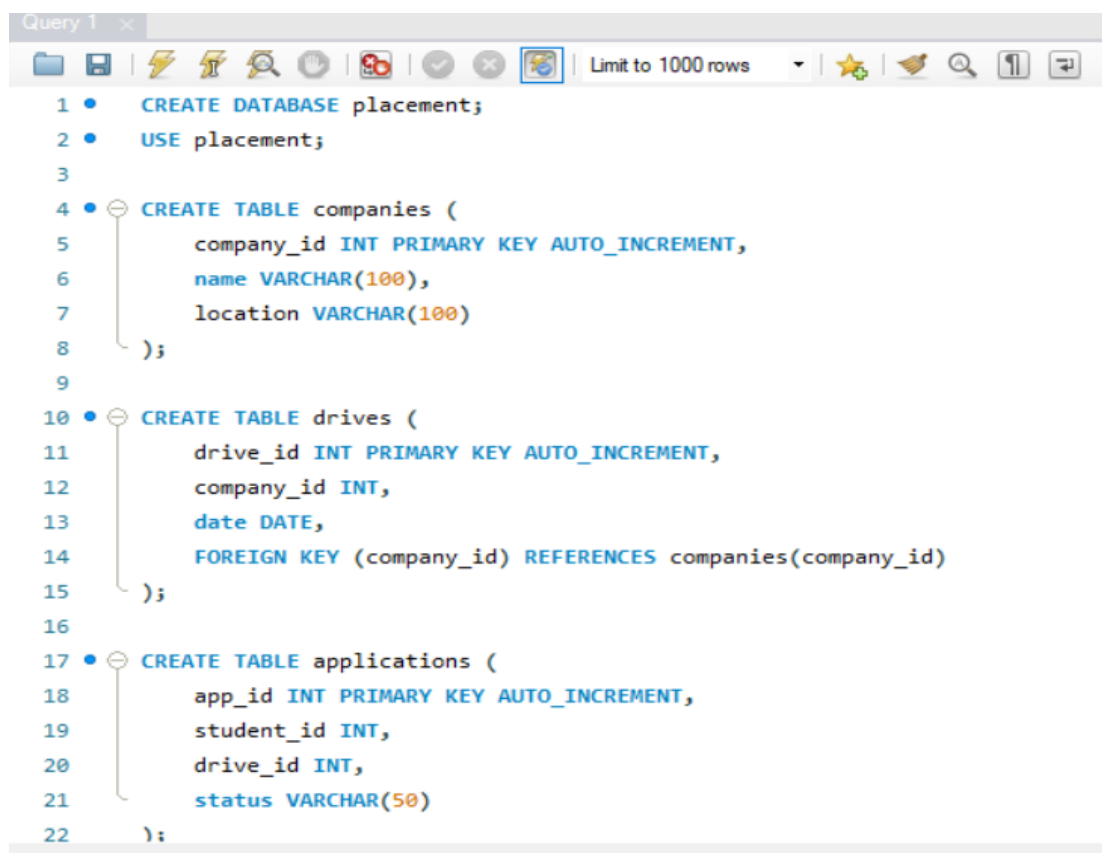
- **Application**

```
CREATE TABLE applications (  
    app_id INT PRIMARY KEY AUTO_INCREMENT,  
    student_id INT,  
    drive_id INT,  
    status VARCHAR(50),  
    FOREIGN KEY (student_id) REFERENCES students(student_id),
```

```
FOREIGN KEY (drive_id) REFERENCES drives(drive_id)  
);
```

Table Relationships:

- **A Company can have multiple Drives (1-to-Many)**
- **A Student can apply to multiple Drives (1-to-Many)**
- **A Drive can receive multiple Applications (1-to-Many)**
- **An Application can have one Interview Slot (1-to-1)**



```
Query 1 x  
Limit to 1000 rows  
1 • CREATE DATABASE placement;  
2 • USE placement;  
3  
4 • CREATE TABLE companies (  
5     company_id INT PRIMARY KEY AUTO_INCREMENT,  
6     name VARCHAR(100),  
7     location VARCHAR(100)  
8 );  
9  
10 • CREATE TABLE drives (  
11     drive_id INT PRIMARY KEY AUTO_INCREMENT,  
12     company_id INT,  
13     date DATE,  
14     FOREIGN KEY (company_id) REFERENCES companies(company_id)  
15 );  
16  
17 • CREATE TABLE applications (  
18     app_id INT PRIMARY KEY AUTO_INCREMENT,  
19     student_id INT,  
20     drive_id INT,  
21     status VARCHAR(50)  
22 );
```

3. DAO Layer Design

- **The DAO layer is responsible for handling all database interactions.**
- **DAO Classes Implemented**
- **CompanyDAO**

```
src > dao > J CompanyDAO.java > CompanyDAO
1 package dao;
2 import java.sql.*;
3 import db.DBConnection;
4
5 public class CompanyDAO {
6
7     public void addCompany(String name, String location) throws Exception {
8         Connection conn = DBConnection.getConnection();
9         String sql = "INSERT INTO companies(name, location) VALUES (?, ?)";
10
11         PreparedStatement ps = conn.prepareStatement(sql);
12         ps.setString(parameterIndex: 1, name);
13         ps.setString(parameterIndex: 2, location);
14         ps.executeUpdate();
15
16         conn.close();
17     }
18 }
```

• DriveDAO

```
src > dao > J DriveDAO.java > DriveDAO
1 package dao;
2 import java.sql.*;
3 import db.DBConnection;
4
5 public class DriveDAO {
6
7     public void addDrive(int companyId, String date) throws Exception {
8         Connection conn = DBConnection.getConnection();
9         String sql = "INSERT INTO drives(company_id, date) VALUES (?, ?)";
10
11         PreparedStatement ps = conn.prepareStatement(sql);
12         ps.setInt(parameterIndex: 1, companyId);
13         ps.setString(parameterIndex: 2, date);
14         ps.executeUpdate();
15
16         conn.close();
17     }
18 }
```

ApplicationDAO

```
src > dao > ApplicationDAO.java > ApplicationDAO
1 package dao;
2 import java.sql.*;
3 import db.DBConnection;
4
5 public class ApplicationDAO {
6
7     public void apply(int studentId, int driveId, String status) throws Exception {
8         Connection conn = DBConnection.getConnection();
9         String sql = "INSERT INTO applications(student_id, drive_id, status) VALUES (?, ?, ?)";
10
11         PreparedStatement ps = conn.prepareStatement(sql);
12         ps.setInt(parameterIndex: 1, studentId);
13         ps.setInt(parameterIndex: 2, driveId);
14         ps.setString(parameterIndex: 3, status);
15
16         ps.executeUpdate();
17         conn.close();
18     }
19 }
```

- **Each DAO class:**
- **Uses JDBC for database interaction**
- **Uses PreparedStatement for security**
- **Contains CRUD operations**

4. CRUD Operations Implementation

CompanyDAO

- **Add Company**
- **View Companies**
- **Update Company**
- **Delete Company**

```
1 package dao;
2 import java.sql.*;
3 import db.DBConnection;
4
5 public class CompanyDAO {
6     // CREATE
7     public void addCompany(String name, String location) throws Exception {
8         Connection conn = DBConnection.getConnection();
9         String sql = "INSERT INTO companies(name, location) VALUES (?, ?)";
10        PreparedStatement ps = conn.prepareStatement(sql);
11        ps.setString(1, name);
12        ps.setString(2, location);
13        ps.executeUpdate();
14
15        conn.close();
16    }
17    // READ
18    public void getAllCompanies() throws Exception {
19        Connection conn = DBConnection.getConnection();
20        String sql = "SELECT * FROM companies";
21        PreparedStatement ps = conn.prepareStatement(sql);
22        ResultSet rs = ps.executeQuery();
23
24        while (rs.next()) {
25            System.out.println(
26                rs.getInt("company_id") + " | " +
27                rs.getString("name") + " | " +
28                rs.getString("location")
29            );
30        }
31
32        conn.close();
33    }
34    // UPDATE
35    public void updateCompany(int id, String newName, String newLocation) throws Exception {
36        Connection conn = DBConnection.getConnection();
37        String sql = "UPDATE companies SET name=?, location=? WHERE company_id=?";
38        PreparedStatement ps = conn.prepareStatement(sql);
39        ps.setString(1, newName);
40        ps.setString(2, newLocation);
41        ps.setInt(3, id);
42
43        ps.executeUpdate();
44        conn.close();
45    }
46    // DELETE
47    public void deleteCompany(int id) throws Exception {
48        Connection conn = DBConnection.getConnection();
49        String sql = "DELETE FROM companies WHERE company_id=?";
50
51        PreparedStatement ps = conn.prepareStatement(sql);
52        ps.setInt(1, id);
53
54        ps.executeUpdate();
55        conn.close();
56    }
57 }
```


DriveDAO

- Create Drive
- View Drives

```
1  package dao;
2
3  import java.sql.*;
4  import db.DBConnection;
5
6  public class DriveDAO {
7
8      // ✅ CREATE
9      public void addDrive(int companyId, String date) throws Exception {
10         Connection conn = DBConnection.getConnection();
11         String sql = "INSERT INTO drives(company_id, date) VALUES (?, ?)";
12
13         PreparedStatement ps = conn.prepareStatement(sql);
14         ps.setInt(1, companyId);
15         ps.setString(2, date);
16
17         ps.executeUpdate();
18         conn.close();
19     }
20
21     // ✅ READ
22     public void getAllDrives() throws Exception {
23         Connection conn = DBConnection.getConnection();
24         String sql = "SELECT * FROM drives";
25
26         PreparedStatement ps = conn.prepareStatement(sql);
27         ResultSet rs = ps.executeQuery();
28
29         while (rs.next()) {
30             System.out.println(
31                 "Drive ID: " + rs.getInt("drive_id") +
32                 " | Company ID: " + rs.getInt("company_id") +
33                 " | Date: " + rs.getString("date")
34             );
35         }
36
37         conn.close();
38     }
39 }
```

ApplicationDAO

- Apply for Drive
- View Applications

```
1 package dao;
2
3 import java.sql.*;
4 import db.DBConnection;
5
6 public class ApplicationDAO {
7
8     // ✅ CREATE (Apply for Drive)
9     public void apply(int studentId, int driveId, String status) throws Exception {
10         Connection conn = DBConnection.getConnection();
11         String sql = "INSERT INTO applications(student_id, drive_id, status) VALUES (?, ?, ?)";
12
13         PreparedStatement ps = conn.prepareStatement(sql);
14         ps.setInt(1, studentId);
15         ps.setInt(2, driveId);
16         ps.setString(3, status);
17
18         ps.executeUpdate();
19         conn.close();
20     }
21
22     // ✅ READ
23     public void getApplications() throws Exception {
24         Connection conn = DBConnection.getConnection();
25         String sql = "SELECT * FROM applications";
26
27         PreparedStatement ps = conn.prepareStatement(sql);
28         ResultSet rs = ps.executeQuery();
29
30         while (rs.next()) {
31             System.out.println(
32                 "App ID: " + rs.getInt("app_id") +
33                 " | Student ID: " + rs.getInt("student_id") +
34                 " | Drive ID: " + rs.getInt("drive_id") +
35                 " | Status: " + rs.getString("status")
36             );
37         }
38
39         conn.close();
40     }
41 }
```

5. JDBC Implementation

The project follows best JDBC practices:

- **Centralized connection handling using a utility class**
- **Usage of Prepared Statements**
- **Proper ResultSet mapping**
- **Separation of SQL from UI layer**

```
src > db > J DBConnection.java > DBConnection > getConnection()
1  package db;
2  import java.sql.*;
3
4  public class DBConnection {
5      public static Connection getConnection() throws Exception {
6          return DriverManager.getConnection(
7              url: "jdbc:mysql://localhost:3306/placement",
8              user: "root",
9              password: "1234"
10         );
11     }
12 }
```

6. Working Flow of the System

System Flow:

User → Main Class → DAO Layer → DBConnection → MySQL Database

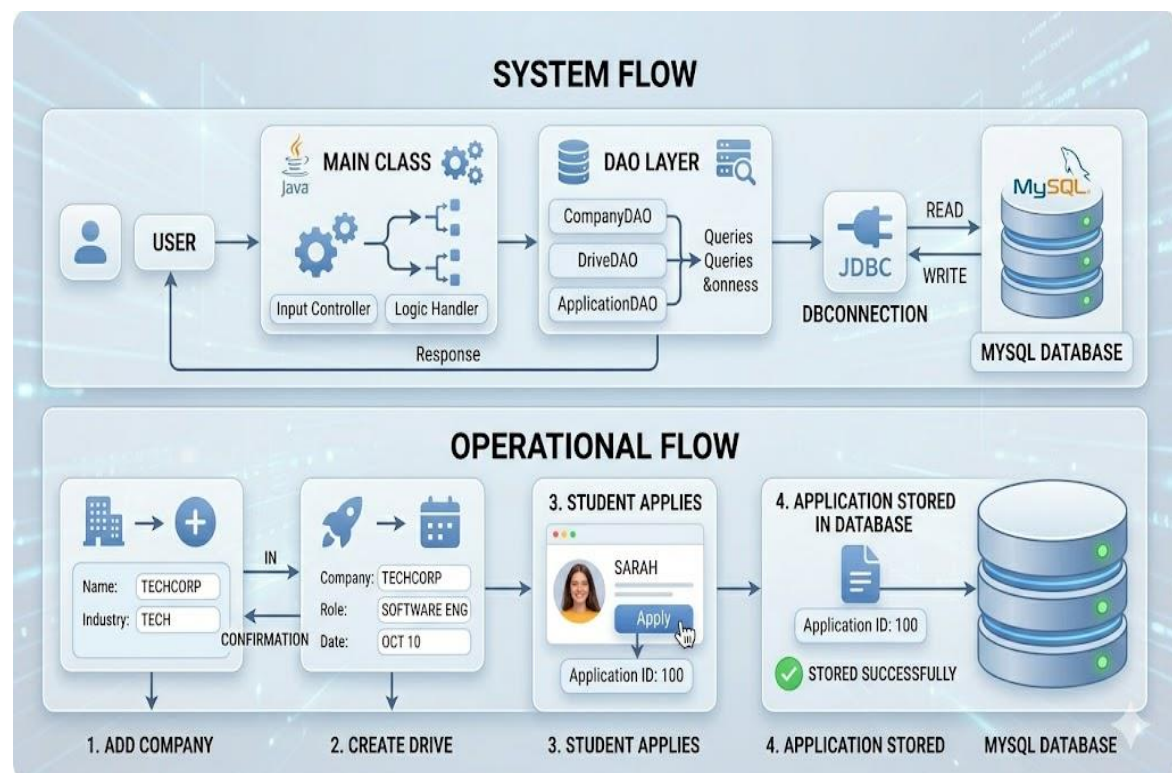
Operational Flow:

1. Add Company

2. Create Drive

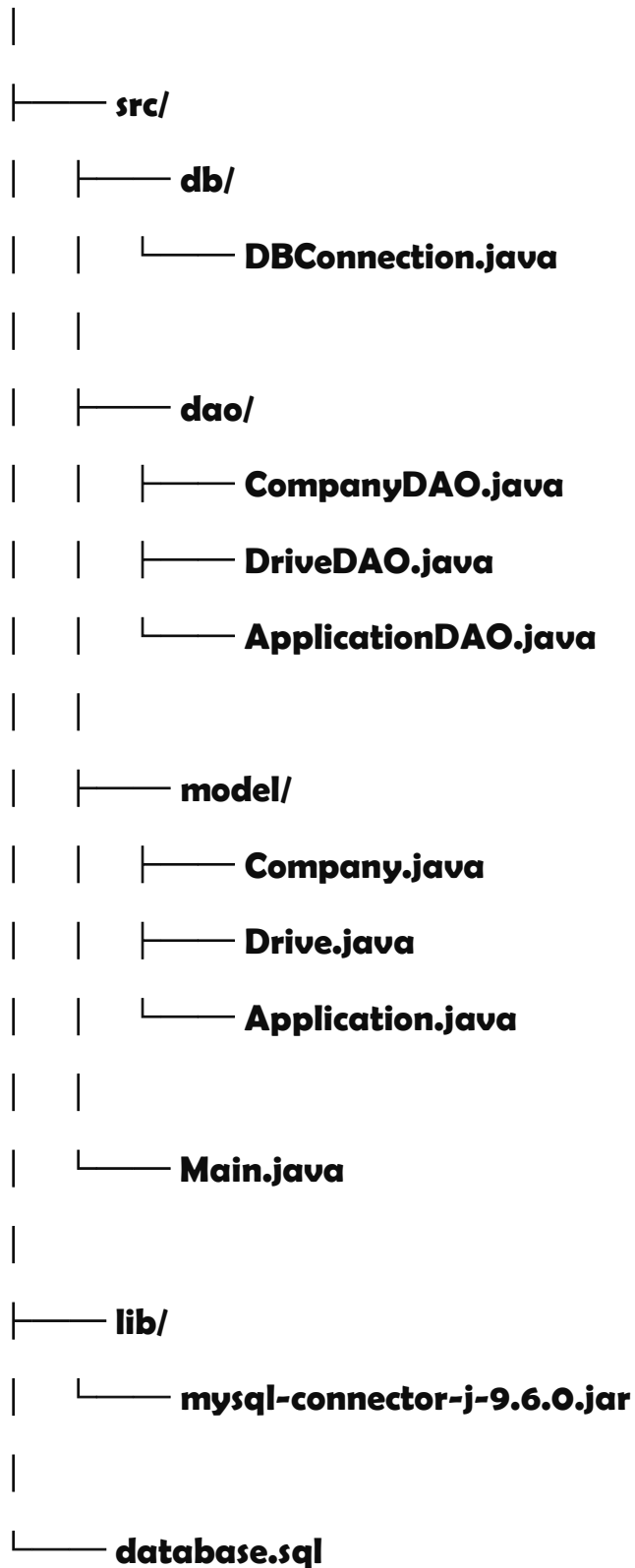
3. Student Applies

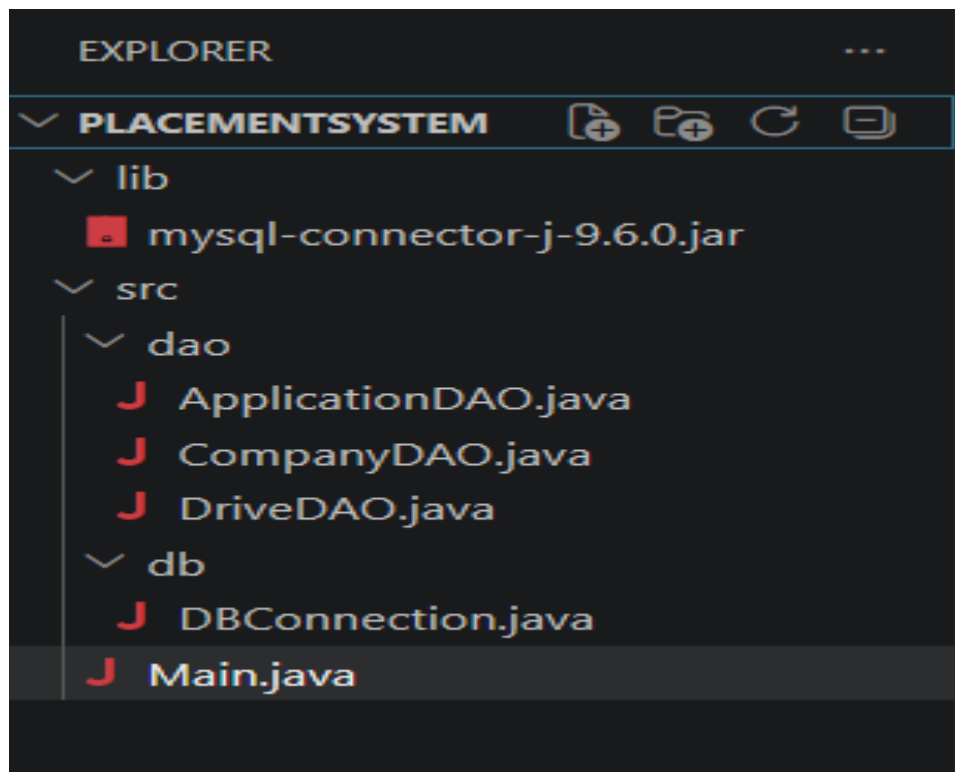
4. Application Stored in Database



7. Project Folder Structure:

PlacementSystem/





8. Design Rules Followed

- **No SQL queries in UI layer**
- **DAO layer strictly handles database**
- **Use of primary and foreign keys**
- **Clean separation of layers**

```
C:\Users\Mayank Tanishk\OneDrive\Desktop\PlacementSystem>javac -cp ".;lib/mysql-connector-j-9.6.0.jar" src\db\DBConnection.java src\dao\*.java src>Main.java

C:\Users\Mayank Tanishk\OneDrive\Desktop\PlacementSystem>java -cp ".;lib/mysql-connector-j-9.6.0.jar;src" Main
Data Inserted Successfully!
```

9. Common Mistakes Avoided

- **No missing primary keys**
- **Proper foreign key relationships**
- **No direct DB access from UI**
- **No scattered SQL queries**

10. Review Checklist

- **Schema matches system design**
- **DAO layer supports CRUD**
- **Database relationships are correct**
- **JDBC implementation works properly**
- **One complete workflow is functional**

```
25 • USE placement;
26
27 • SELECT * FROM companies;
28 • SELECT * FROM drives;
29 • SELECT * FROM applications;
```

Result Grid |  Filter Rows:

	company_id	name	location
▶	1	Google	Bangalore
•	NULL	NULL	NULL